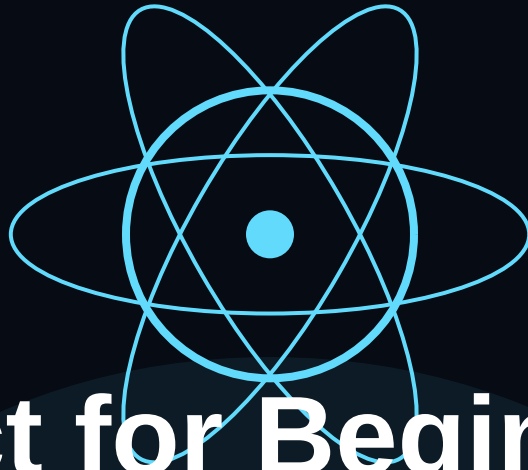




React for Beginners

The 80/20 Course — Learn What Actually Matters

For developers who know HTML, CSS & JavaScript
and want to be internship-ready FAST.

10

Lessons

60+

Exercises

1

Final Project

JSX

Components

Props

State

Events

Forms

useState

useEffect

Router

API Fetching

01 JSX — Writing HTML in JavaScript

01-jsx/

02 Components — LEGO Blocks of the Web

02-components/

03 Props — Passing Info Between Components

03-props/

04 State — React's Memory

04-state/

05 Events — Making Things Interactive

05-events/

06 Forms — Collecting User Input

06-forms/

07 useState — The Full Story

07-useState/

08 useEffect — Side Effects & Lifecycle

08-useEffect/

09 React Router — Multi-Page Apps

09-router/

10 API Fetching — Real Data, Real Apps

10-api-fetching/

■ After Lessons

→ React Mastery Roadmap

→ Final Capstone Project — Movie Night Planner

LESSON 01

JSX — Writing HTML in JavaScript

1. Why JSX Exists

Before React, building a dynamic web page was painful. You had to write JavaScript separately from your HTML, find elements with `document.getElementById()`, then manipulate them one by one. It was like building a car by bolting parts together from different rooms.

■ ANALOGY

Imagine you're writing a birthday card, but you have to write the words in one book and the decorations in another book — then glue them together later. JSX lets you write BOTH in one place.

JSX stands for **JavaScript XML**. It lets you write HTML-looking code *inside* your JavaScript. React then converts it into real JavaScript behind the scenes.

Without JSX you'd have to write this to create a heading:

■ Without JSX

```
// Without JSX – ugly and hard to read
const element = React.createElement(
  'h1',
  { className: 'title' },
  'Hello World'
);
```

With JSX you write this:

■ With JSX

```
// With JSX – looks like HTML, easy to read!
const element = <h1 className='title'>Hello World</h1>;
```

2. Mental Model

■ MENTAL MODEL

JSX is like a magic translator.
You write it like HTML. React reads it like JavaScript.
You get the best of both worlds.

Think of JSX as a template. You describe what the page should *look like*, and React figures out how to actually build it. You're the architect — JSX is your blueprint.

3. Industry Relevance

■ INDUSTRY

■ USAGE LEVEL: 100% — Every single React file uses JSX.

You will write JSX every day as a React developer.

It appears in every component in every React app at Netflix, Airbnb, Facebook, Stripe, and every startup that uses React.

4. Smallest Possible Example

App.jsx — Your first JSX

```
// App.jsx
function App() {
  const name = 'Alex';
  const age = 17;

  return (
    <div>
      <h1>Hello, {name}!</h1>
      <p>You are {age} years old.</p>
      <p>In 3 years you will be {age + 3}.</p>
    </div>
  );
}
```

Notice the curly braces `{}`. Whenever you want to use real JavaScript inside JSX, you wrap it in `{}`. Think of them as 'JavaScript portals'.

5. The 5 JSX Rules You MUST Know

Rule 1: One Parent Element

JSX must return ONE parent element. If you need multiple elements, wrap them in a or use <> (called a Fragment).

```
// ■ WRONG — two siblings at the top level
return <h1>Hi</h1><p>World</p>;

// ■ RIGHT — wrapped in a fragment
return (
  <>
    <h1>Hi</h1>
    <p>World</p>
  </>
);
```

Rule 2: className, not class

HTML uses 'class', but 'class' is a reserved word in JavaScript. In JSX, always use className.

```
// ■ WRONG
<div class="card">...

// ■ RIGHT
<div className="card">...
```

Rule 3: Close Every Tag

In HTML, br and img tags are fine unclosed. In JSX, everything must close.

```
// ■ WRONG
<br>


// ■ RIGHT
<br />

```

Rule 4: Curly Braces for JavaScript

Use {} to escape into JavaScript. You can use variables, expressions, function calls — anything that returns a value.

```
const color = "red";
return <p style={{ color: color }}>I am red!</p>;
// Note: style takes a JS object, so double {{ }}
```

Rule 5: camelCase for Attributes

HTML attributes like `onclick` and `tabindex` become `onClick` and `tabIndex` in JSX.

```
// HTML:   <button onclick="go()">
// JSX:
<button onClick={go}>Click me</button>
```

6. Common Beginner Mistakes

■ COMMON MISTAKE

Forgetting to close self-closing tags like `<img>` and `<input>`.

■ COMMON MISTAKE

Using `'class'` instead of `'className'`.

■ COMMON MISTAKE

Trying to put an if statement directly inside JSX — use a ternary (`condition ? a : b`) instead.

■ COMMON MISTAKE

Returning multiple elements without wrapping them.

■ COMMON MISTAKE

Forgetting curly braces when inserting variables — writing `{name}` not `name`.

7. ■■ Mini Project — Personal Profile Card

■■ MINI PROJECT

Build a profile card using only JSX.

Include: your name, a fun fact, your skills, and a 'hire me' button.

Practice: `className`, curly braces `{}`, self-closing tags, fragments.

Time: 10–20 minutes. No backend needed.

ProfileCard.jsx

```
function ProfileCard() {
  const name = 'Taylor Swift'; // change this!
  const role = 'Frontend Dev';
  const skills = ['React', 'CSS', 'JavaScript'];

  return (
    <div className='card'>
      <h1>{name}</h1>
      <p className='role'>{role}</p>
      <ul>
        {skills.map(skill => <li key={skill}>{skill}</li>)}
      </ul>
      <button>Hire Me!</button>
    </div>
  );
}
```

8. Exercises

[Easy] Create a JSX element that displays today's date using JavaScript's new `Date().toString()`.

[Easy] Write a JSX component that shows a list of 5 of your favourite movies.

[Easy] Make a JSX 'badge' with your name and a background color using inline style.

[Medium] Create a JSX receipt component: item name, quantity, price, and calculated total.

[Medium] Build an 'error message' JSX component that takes a message and shows it in red only when message is not empty (use ternary).

[Challenge] Build a mini-resume in JSX: name, title, 3 skills, 2 projects (each with a title and description). Make it look good with `className`.

9. Interview Questions

Q: What is JSX?

A: JSX is a syntax extension for JavaScript that lets you write HTML-like markup inside JavaScript. React transforms it into `React.createElement()` calls.

Q: Why do we use `className` instead of `class` in JSX?

A: Because 'class' is a reserved keyword in JavaScript. JSX is still JavaScript under the hood, so it uses `className` to avoid conflicts.

Q: What are the curly braces `{}` used for in JSX?

A: They let you embed any JavaScript expression inside JSX. Variables, function calls, math — anything that produces a value.

Q: What is a Fragment in React?

A: A Fragment (`<>`) lets you group multiple JSX elements without adding an extra DOM node. It keeps the HTML clean.

10. Real World Connection

■ INDUSTRY

At companies like Airbnb and Stripe, JSX is the foundation of every UI. A product page on Airbnb is made of dozens of JSX components all working together. When you look at a hotel listing, the photo gallery, the price badge, the booking form — all written in JSX, all assembled from pieces.

11. Lesson Summary

■ KEY TAKEAWAY

JSX lets you write HTML-looking code inside JavaScript.

■ KEY TAKEAWAY

Always return ONE parent element (use `<>` if needed).

■ KEY TAKEAWAY

Use `{}` to embed JavaScript expressions inside JSX.

■ KEY TAKEAWAY

Use `className`, not `class`. Close all tags. Use camelCase attributes.

■ KEY TAKEAWAY

JSX is NOT HTML — it's JavaScript in disguise.

■ *Remember forever: **JSX = JavaScript + HTML married together.** The curly braces `{}` are where you bring JavaScript to the party.*

LESSON 02

Components — LEGO Blocks of the Web

1. Why Components Exist

Imagine building a house by carving it from a single rock. If you want to change a window, you'd have to re-carve the whole rock. Terrible.

Now imagine building with LEGO blocks. Each block is independent. You can swap a blue block for a red one without touching everything else. That's what components do for websites.

■ ANALOGY

LEGO blocks = Components.

Each block (component) is reusable, independent, and snaps together.

The whole website is just components stacked inside components.

Before components, every webpage was one giant wall of HTML. Change one thing → break everything. Components let us break the UI into small, manageable, reusable pieces.

2. Mental Model

■ MENTAL MODEL

A component is just a function that returns JSX.

Call the function → get a piece of UI.

Use it anywhere, as many times as you want.

Components can live inside other components.

3. Industry Relevance

■ INDUSTRY

■ USAGE LEVEL: 100% — Components ARE React.

Every React app is a tree of components.

Junior devs spend 90% of their time creating and connecting components.

At Twitter/X, the tweet, the sidebar, the nav bar — all separate components.

4. Anatomy of a Component

Your First Component

```
// 1. Define a component (it's just a function!)
function Greeting() {
  return <h1>Hello, World!</h1>;
}

// 2. Use it like an HTML tag
function App() {
  return (
    <div>
      <Greeting />
      <Greeting />
      <Greeting />
    </div>
  );
}
// Result: three 'Hello, World!' headings
```

■ TIP

Component names MUST start with a capital letter.
<Greeting /> is a component. <greeting /> is an HTML tag.
React tells them apart by the capital letter — don't forget this!

5. Component Tree

React apps are organized as a tree — like a family tree, but for components:

Component Tree (ASCII diagram)

```
      App
     /  \
  Header MainContent
   /  \   \
 Logo Nav ProductCard ProductCard ProductCard
```

The top component (App) is the 'root'. Everything branches down from there. Parent components contain child components.

6. Reusing Components

Reusing a Component

```
// One definition – used three times!
function MovieCard() {
  return (
    <div className='card'>
      <h2>Movie Title</h2>
      <p>Rating: ■■■■</p>
    </div>
  );
}

function App() {
  return (
    <div>
      <MovieCard />
      <MovieCard />
      <MovieCard />
    </div>
  );
}
```

7. Common Beginner Mistakes

■ COMMON MISTAKE

Lowercase component names: <greeting /> doesn't work — must be <Greeting />.

■ COMMON MISTAKE

Defining a component inside another component (big performance problem).

■ COMMON MISTAKE

Forgetting to export the component — other files can't use it!

■ COMMON MISTAKE

Using a component before defining it (JSX is JS, order matters sometimes).

8. Exporting Components

Exporting & Importing Components

```
// Card.jsx – define and export
export default function Card() {
  return <div className='card'>I am a card</div>;
}

// App.jsx – import and use
import Card from './Card';

function App() {
  return <Card />;
}
```

9. ■■ Mini Project — LEGO Website Builder

■■ MINI PROJECT

Build a simple webpage using only components.

Create: <Header />, <Hero />, <FeatureCard />, <Footer />

Use <FeatureCard /> at least 3 times.

Put them all together in <App />.

Goal: feel how components snap together like LEGO.

LEGO Website Builder Starter

```
function Header() {
  return <nav><h1>My App</h1></nav>;
}

function FeatureCard() {
  return <div className='feature'>■ Feature here</div>;
}

function Footer() {
  return <footer>© 2024 My App</footer>;
}

function App() {
  return (
    <>
      <Header />
      <FeatureCard />
      <FeatureCard />
      <FeatureCard />
      <Footer />
    </>
  );
}
```

10. Exercises

[Easy] Create a component that renders a styled button with the text 'Click Me'.

[Easy] Create an component that shows a circle with initials (like 'JS' for John Smith).

[Easy] Create a component that shows a label in a coloured pill shape.

[Medium] Build a component that contains a logo and 3 navigation links.

[Medium] Create a component with a plan name, price, and list of 3 features. Use it 3 times for Basic, Pro, and Enterprise.

[Challenge] Build a full landing page using at least 6 different components. Give each its own file and import them into App.jsx.

11. Interview Questions

Q: What is a React component?

A: A component is a JavaScript function that returns JSX — a piece of UI. Components are reusable, independent building blocks of a React application.

Q: What is the difference between a parent and child component?

A: A parent component renders (contains) a child component. Data flows from parent to child via props. The App component is typically the top-level parent.

Q: Why must component names start with a capital letter?

A: React uses the capital letter to distinguish custom components from built-in HTML elements. `isHTML` would be a React component.

12. Lesson Summary

■ KEY TAKEAWAY

Components are functions that return JSX — they're the building blocks of React.

■ KEY TAKEAWAY

Component names must start with a CAPITAL letter.

■ KEY TAKEAWAY

Components can be reused as many times as you want.

■ KEY TAKEAWAY

Components can contain other components (parent → child).

■ KEY TAKEAWAY

Each component lives in its own file and is exported/imported.

LESSON 03

Props — Passing Info Between Components

1. Why Props Exist

Remember the component from Lesson 02? Every card showed the exact same movie. That's useless in real life!

■ ANALOGY

Props are like delivery packages.
Imagine the parent component is Amazon's warehouse.
It ships packages (props) to child components.
Each package has information written on it: name, rating, image...
The child component opens the package and uses what's inside.

Without props, every would show the same movie forever. Props let us **customize** each instance of a component with different data.

2. Mental Model

■ MENTAL MODEL

Props are the arguments you pass to a component.
Just like a function takes parameters, a component takes props.
Parent writes info on the package. Child unwraps and uses it.
Data ALWAYS flows one way: parent → child (never the reverse).

3. Smallest Possible Example

Your First Props

```
// Child component – receives props
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}

// Parent component – sends props
function App() {
  return (
    <div>
      <Greeting name='Alice' />
      <Greeting name='Bob' />
      <Greeting name='Carlos' />
    </div>
  );
}
// Output: Hello, Alice! Hello, Bob! Hello, Carlos!
```

4. Destructuring Props (The Professional Way)

Instead of writing `props.name` and `props.age` everywhere, you can unpack props at the top using destructuring:

Destructured Props

```
// Instead of this:
function Card(props) {
  return <h2>{props.title} – {props.rating}</h2>;
}

// Write this (cleaner!):
function Card({ title, rating, image }) {
  return (
    <div className='card'>
      <img src={image} alt={title} />
      <h2>{title}</h2>
      <p>Rating: {rating}</p>
    </div>
  );
}

// Usage:
<Card
  title='Dune'
  rating='9.0'
  image='dune.jpg'
/>
```


5. Default Props

What if someone forgets to pass a prop? Use default values to protect your component:

Default Props

```
function Button({ text = 'Click Me', color = 'blue' }) {
  return (
    <button style={{ background: color }}>
      {text}
    </button>
  );
}

// Uses defaults:
<Button />

// Overrides defaults:
<Button text='Submit' color='green' />
```

6. Props Can Be Anything

Props Can Be Any Data Type

```
// String:
<Profile name='Alice' />

// Number (no quotes!):
<Profile age={25} />

// Boolean:
<Button disabled={true} />

// Array:
<List items={['React', 'CSS', 'JS']} />

// Object:
<Profile user={{ name: 'Alice', age: 25 }} />

// Function:
<Button onClick={handleClick} />
```

■ TIP

Strings use quotes: name='Alice'

Everything else uses curly braces: age={25}, items={myArray}

7. Common Beginner Mistakes

■ COMMON MISTAKE

Trying to modify props inside the child — props are READ ONLY. Never change them!

■ COMMON MISTAKE

Forgetting curly braces for non-string props: `age='25'` gives a string, `age={25}` gives a number.

■ COMMON MISTAKE

Passing props to the wrong component — check your component tree.

■ COMMON MISTAKE

Misspelling prop names (`propss`, `naem`) — JavaScript won't warn you about typos in prop names.

8. ■■ Mini Project — Product Card Generator

■■ MINI PROJECT

Build a reusable `<ProductCard />` component.

Props it should accept: `name`, `price`, `category`, `inStock` (boolean), `image`.

Show a 'Sold Out' badge when `inStock` is false.

Use it 4 times in `App.jsx` with different products.

Time: 20–30 minutes.

ProductCard.jsx

```
function ProductCard({ name, price, category, inStock, image }) {
  return (
    <div className='product-card'>
      <img src={image} alt={name} />
      <span className='category'>{category}</span>
      <h3>{name}</h3>
      <p>${price}</p>
      {inStock ? (
        <button>Add to Cart</button>
      ) : (
        <span className='sold-out'>Sold Out</span>
      )}
    </div>
  );
}
```

9. Exercises

[Easy] Create a component that takes a label prop and renders a coloured badge.

[Easy] Create a component that takes a rating (1-5) and shows that many ■.

[Easy] Create a that takes name and src props. Show the image if src exists, otherwise show initials.

[Medium] Build a component with title, author, date, and excerpt props. Style it nicely.

[Medium] Create a that takes value (0-100) and color props. Display a bar that fills proportionally.

[Challenge] Build a that takes a columns array and rows array of objects, and renders a styled HTML table dynamically from them.

10. Interview Questions

Q: What are props in React?

A: Props (short for properties) are the way parent components pass data to child components. They are read-only — a child cannot modify its own props.

Q: What does 'one-way data flow' mean?

A: Data in React always flows from parent to child through props, never the other way around. This makes apps predictable and easier to debug.

Q: Can you change props inside a component?

A: No. Props are immutable (read-only) from the child's perspective. If you need to change data, you use state (the next lesson!).

11. Lesson Summary

■ KEY TAKEAWAY

Props are how parent components send data to child components.

■ KEY TAKEAWAY

Props are read-only — never modify them inside the child.

■ KEY TAKEAWAY

Destructure props for cleaner, more readable code.

■ KEY TAKEAWAY

Use default values to make your components bulletproof.

■ KEY TAKEAWAY

Data flows ONE WAY: parent → child. Always.

LESSON 04

State — React's Memory

1. Why State Exists

Props are great for static data. But what happens when things need to **change**? A like counter that goes up. A dropdown that opens and closes. A form field where the user types.

■ ANALOGY

Think of State as a whiteboard inside a component.
The component can write on it, erase it, and re-read it anytime.
When the whiteboard changes, React automatically redraws the screen.
Props are messages FROM outside. State is the component's OWN memory.

Without state, your React apps would be completely static — like a photograph. With state, they become dynamic — like a video.

2. Mental Model

■ MENTAL MODEL

State = a component's own private memory.
When state changes, React re-renders (redraws) the component.
You never update state directly — always use the setter function.
Think: 'what things can CHANGE on screen?' → those are state.

3. useState — The State Hook

You declare state with the useState hook. It gives you two things: the current value, and a function to update it:

Counter with State

```
import { useState } from 'react';

function Counter() {
  // [currentValue, setterFunction] = useState(initialValue)
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(count - 1)}>-1</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

■ TIP

The naming convention is: [thing, setThing]

count → setCount

isOpen → setIsOpen

username → setUsername

4. How React Re-renders

Re-render Flow

State changes	React re-renders	Screen updates
setCount(count+1) →	Counter() runs again →	New count shown

It's automatic! You update state, React handles the rest.

5. State with Different Data Types

State Can Hold Any Type

```
// Number
const [likes, setLikes] = useState(0);

// String
const [name, setName] = useState('');

// Boolean
const [isOpen, setIsOpen] = useState(false);

// Array
const [todos, setTodos] = useState([]);

// Object
const [user, setUser] = useState({ name: '', age: 0 });
```

6. Common Beginner Mistakes

■ COMMON MISTAKE

Setting state directly: `count = count + 1`; ■ — React won't know it changed!

■ COMMON MISTAKE

Calling `useState` outside a component — hooks only work inside React functions.

■ COMMON MISTAKE

Expecting the new state value immediately after calling `setState` (it updates asynchronously).

■ COMMON MISTAKE

Mutating state objects/arrays directly instead of creating new copies.

7. ■■ Mini Project — Mood Tracker

■■ MINI PROJECT

Build a Mood Tracker app.

State to track: current mood (string), mood history (array), streak (number).

Moods: ■ Happy, ■ Neutral, ■ Sad, ■ Excited, ■ Angry.

Click a mood → update state → show history below.

Time: 20–30 minutes.

MoodTracker.jsx

```
function MoodTracker() {
  const [currentMood, setCurrentMood] = useState('');
  const [history, setHistory] = useState([]);

  const moods = ['😄', '😐', '😞', '😡', '😭'];

  function selectMood(mood) {
    setCurrentMood(mood);
    setHistory([...history, mood]); // spread = new array
  }

  return (
    <div>
      <h1>How are you feeling?</h1>
      {moods.map(m => (
        <button key={m} onClick={() => selectMood(m)}>{m}</button>
      ))}
      <p>Current mood: {currentMood}</p>
      <h2>History:</h2>
      <p>{history.join(' ')}</p>
    </div>
  );
}
```

8. Exercises

[Easy] Build a toggle button: clicking it switches the text between 'ON' and 'OFF'.

[Easy] Create a 'colour picker' with 5 buttons. Clicking each changes the background colour using state.

[Easy] Build a step counter: buttons for +1 (walk), +5 (jog), +10 (run) and a reset.

[Medium] Create a score tracker for 2 players. Each has their own +1 and -1 button. The higher score name turns green.

[Medium] Build a shopping cart item counter for 3 products. Show the total count across all products.

[Challenge] Build a basic flashcard app: show a question, button to reveal the answer, buttons to mark 'Got it' or 'Try again'. Track score.

9. Interview Questions

Q: What is state in React?

A: State is a component's internal memory — data that the component owns and can change over time. When state changes, React automatically re-renders the component.

Q: What is the difference between state and props?

A: Props come from outside (parent) and are read-only. State is internal and can be changed by the component itself using a setter function.

Q: Why can't you set state directly (like `count = count + 1`)?

A: React uses the setter function (`setCount`) to detect changes and trigger a re-render. Modifying the variable directly bypasses React's tracking system — the screen won't update.

10. Lesson Summary

■ **KEY TAKEAWAY**

State is a component's private memory — data that can change.

■ **KEY TAKEAWAY**

Use `useState(initialValue)` to declare state.

■ **KEY TAKEAWAY**

Always update state with the setter function (`setX`), never directly.

■ **KEY TAKEAWAY**

When state changes → React automatically re-renders the component.

■ **KEY TAKEAWAY**

Ask yourself: 'What changes on screen?' → that's your state.

LESSON 05

Events — Making Things Interactive

1. Why Events Exist

A webpage without events is a brochure — you can look, but not touch. Events are what make things *happen* when users interact. Clicking, typing, hovering, submitting — all events.

■ ANALOGY

Events are like doorbells.

You press the button (click/type/hover) → the bell rings (event fires)

→ someone answers (your handler function runs).

The event system is React's way of listening and responding.

2. Event Handlers in React

Basic Event Handler

```
// In HTML: onclick='doSomething()'
// In React: onClick={doSomething} (camelCase, no quotes, no ())

function App() {
  function handleClick() {
    alert('You clicked me!');
  }

  return <button onClick={handleClick}>Click Me</button>;
}
```

■ TIP

Pass the function REFERENCE, not the function CALL.

onClick={handleClick} ■ — passes the function itself

onClick={handleClick()} ■ — calls it immediately on render!

3. Common Events You'll Use

Event	When it fires	Common element
onClick	User clicks an element	button, div, anything
onChange	Input value changes	input, select, textarea
onSubmit	Form is submitted	form element

onMouseEnter	Mouse hovers over element	any element
onMouseLeave	Mouse leaves an element	any element
onKeyDown	User presses a key	input, document
onFocus	Element gets keyboard focus	input, button
onBlur	Element loses keyboard focus	input, button

4. The Event Object

Every event handler automatically receives an event object with information about what happened:

The Event Object

```
function App() {
  function handleChange(event) {
    console.log(event.target.value); // what the user typed
    console.log(event.target.name); // the input's name attribute
  }

  function handleClick(event) {
    event.preventDefault(); // stop default browser behaviour
    console.log('clicked at', event.clientX, event.clientY);
  }

  return (
    <div>
      <input onChange={handleChange} name='email' />
      <a href='#' onClick={handleClick}>Link</a>
    </div>
  );
}
```

5. Passing Arguments to Handlers

Sometimes you need to pass extra data to a handler. Use an arrow function:

Passing Arguments to Handlers

```
function App() {
  function handleDelete(id) {
    console.log('Delete item', id);
  }

  return (
    <div>
      {/* Wrap in arrow function to pass arguments */}
      <button onClick={() => handleDelete(1)}>Delete Item 1</button>
      <button onClick={() => handleDelete(2)}>Delete Item 2</button>
      <button onClick={() => handleDelete(3)}>Delete Item 3</button>
    </div>
  );
}
```

6. Common Beginner Mistakes

■ COMMON MISTAKE

`onClick={handleClick()}` — adds `()` which CALLS the function immediately. Remove the parentheses!

■ COMMON MISTAKE

Not using `event.preventDefault()` on form submissions — the page refreshes!

■ COMMON MISTAKE

Forgetting that `onChange` on an `<input>` fires on every keystroke.

■ COMMON MISTAKE

Trying to use event outside the handler — it's only available inside the function.

7. ■■ Mini Project — Click Counter

■■ MINI PROJECT

Build a multi-event interactive app:

- Button that counts clicks
- Hover to see a hidden message
- Input that shows what you type in real-time
- Double-click to reset the counter

Time: 15–25 minutes.

InteractiveCard.jsx

```
function InteractiveCard() {
  const [count, setCount] = useState(0);
  const [hovered, setHovered] = useState(false);
  const [text, setText] = useState('');

  return (
    <div
      onMouseEnter={() => setHovered(true)}
      onMouseLeave={() => setHovered(false)}
    >
      {hovered && <p>■ You found me!</p>}
      <button
        onClick={() => setCount(c => c + 1)}
        onDoubleClick={() => setCount(0)}
      >
        Clicked {count} times
      </button>
      <input
        onChange={e => setText(e.target.value)}
        placeholder='Type something...'
      />
      <p>You typed: {text}</p>
    </div>
  );
}
```

8. Exercises

[Easy] Build a 'spoiler' button: clicking it reveals/hides a hidden text message.

[Easy] Create a hover-to-zoom card: when you hover over it, the card becomes bigger using state + inline style.

[Easy] Build a character counter input: shows how many characters the user has typed.

[Medium] Create a 'copy to clipboard' button that copies text and shows 'Copied!' for 2 seconds.

[Medium] Build a keyboard-driven counter: pressing ArrowUp increases count, ArrowDown decreases it.

[Challenge] Build a 'whack-a-mole' game: a grid of 9 cells, one randomly highlights, user must click it before it moves.

9. Interview Questions

Q: How are React event handlers different from HTML event handlers?

A: React uses camelCase (onClick vs onclick), passes function references not strings, and uses synthetic events that work consistently across all browsers.

Q: What is event.preventDefault() used for?

A: It stops the browser's default behaviour for an event, most commonly used to prevent a form from refreshing the page on submit.

Q: How do you pass arguments to an event handler?

A: Wrap the handler in an arrow function: `onClick={() => handleDelete(id)}`. This creates a new function that calls your handler with the argument when clicked.

10. Lesson Summary

■ **KEY TAKEAWAY**

Events make your app respond to user actions (clicks, typing, hovering).

■ **KEY TAKEAWAY**

React uses camelCase event names: `onClick`, `onChange`, `onSubmit`.

■ **KEY TAKEAWAY**

Pass function references, NOT function calls: `onClick={fn}` not `onClick={fn()}`.

■ **KEY TAKEAWAY**

The event object contains info about what happened (`e.target.value`, etc.).

■ **KEY TAKEAWAY**

Use arrow functions in JSX when you need to pass arguments to handlers.

LESSON 06

Forms — Collecting User Input

1. Why Forms Are Special in React

Forms are the #1 way users give you data — login, signup, search, checkout. In regular HTML, forms submit to a server and refresh the page. In React, we take full control of the form.

■ ANALOGY

A React form is like a smart notepad.

Every letter the user types is immediately saved in state (the notepad).

When they submit, you have ALL the data ready instantly.

No page refresh. No waiting. You're in complete control.

2. Controlled Components — The React Way

A **controlled input** is one where React controls the value. The input's value always matches a state variable. This gives you real-time access to what the user types.

Controlled Login Form

```
import { useState } from 'react';

function LoginForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  function handleSubmit(event) {
    event.preventDefault(); // CRITICAL: stops page refresh
    console.log('Logging in:', email, password);
    // send to API here
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type='email'
        value={email}
        onChange={e => setEmail(e.target.value)}
        placeholder='Email'
      />
      <input
        type='password'
        value={password}
        onChange={e => setPassword(e.target.value)}
        placeholder='Password'
      />
      <button type='submit'>Login</button>
    </form>
  );
}
```

3. Handling Multiple Inputs Efficiently

With many fields, a single handler using `e.target.name` saves you from writing a handler for every field:

Multi-field Form with One Handler

```
function SignupForm() {
  const [form, setForm] = useState({
    name: '', email: '', age: '', role: 'student'
  });

  function handleChange(e) {
    const { name, value } = e.target;
    setForm(prev => ({ ...prev, [name]: value }));
  }

  return (
    <form>
      <input name='name' value={form.name} onChange={handleChange} />
      <input name='email' value={form.email} onChange={handleChange} />
      <input name='age' value={form.age} onChange={handleChange} />
      <select name='role' value={form.role} onChange={handleChange}>
        <option value='student'>Student</option>
        <option value='pro'>Professional</option>
      </select>
    </form>
  );
}
```

4. Basic Form Validation

Form with Validation

```
function ContactForm() {
  const [name, setName] = useState('');
  const [error, setError] = useState('');

  function handleSubmit(e) {
    e.preventDefault();
    if (name.trim() === '') {
      setError('Name cannot be empty!');
      return;
    }
    setError('');
    alert('Submitted: ' + name);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={name} onChange={e => setName(e.target.value)} />
      {error && <p className='error'>{error}</p>}
      <button type='submit'>Send</button>
    </form>
  );
}
```

5. Common Beginner Mistakes

■ COMMON MISTAKE

Forgetting `event.preventDefault()` — the page refreshes and you lose all form data!

■ COMMON MISTAKE

Not having `value={state}` on the input — makes it 'uncontrolled' and unpredictable.

■ COMMON MISTAKE

Using the wrong event: forms use `onSubmit` on the `<form>`, not `onClick` on the button.

■ COMMON MISTAKE

Mutating the state object directly: `setForm({ ...form, name: val })` not `form.name = val`.

6. ■■ Mini Project — Contact Form

■ MINI PROJECT

Build a full contact form with: name, email, subject, message fields.

Add validation: all fields required, email must contain @.

Show a success message after submission.

Clear the form after submission.

Time: 25–35 minutes.

7. Exercises

[Easy] Build a username input that shows a live preview: 'Your username will be: {value}'.

[Easy] Create a dropdown + display: a with 5 colours; selecting one changes the background colour.

[Easy] Build a 'word counter' textarea that shows how many words the user has typed.

[Medium] Create a newsletter signup form with email validation (must include @ and .com).

[Medium] Build a 'create a post' form with title (max 80 chars), body, and category. Show remaining char count.

[Challenge] Build a multi-step form: Step 1: name & email. Step 2: role & skills (checkboxes). Step 3: review and submit. Show a progress bar.

8. Interview Questions

Q: What is a controlled component?

A: A controlled component is an input whose value is driven by React state. Every change updates state via onChange, and the state is fed back to the input via the value prop.

Q: Why do we need event.preventDefault() in form handlers?

A: By default, submitting a form causes the browser to send a GET/POST request and refresh the page. preventDefault() stops this so we can handle submission with JavaScript.

Q: What is the spread operator used for in form state updates?

A: setForm(prev => ({ ...prev, [name]: value })) creates a NEW object with all existing fields plus the updated one. This is how we safely update one field of a multi-field state object.

9. Lesson Summary

■ KEY TAKEAWAY

React forms use controlled components — input value tied to state.

■ KEY TAKEAWAY

ALWAYS use event.preventDefault() on form submission.

■ **KEY TAKEAWAY**

Use `onChange` to update state on every keystroke.

■ **KEY TAKEAWAY**

For multiple fields, use one state object and one `handleChange` function.

■ **KEY TAKEAWAY**

Validate before submitting and show errors in the UI using state.

LESSON 07

useState — The Full Story

1. Going Deeper with useState

You already met useState in Lesson 04. Now we go deeper — the patterns that real developers use every day in production code.

■ MENTAL MODEL

useState is like a special variable.

Normal variable: changes don't update the screen.

useState variable: ANY change triggers a re-render.

This is the core magic of React.

2. Functional Updates — The Safe Pattern

When new state depends on old state, use a function inside the setter. This guarantees you get the *latest* value even in async situations:

Functional Update Pattern

```
// ■ Can be stale in async code
setCount(count + 1);

// ■ Always uses the latest value
setCount(prev => prev + 1);

// Real example – adding to a list
setItems(prev => [...prev, newItem]);

// Removing from a list
setItems(prev => prev.filter(item => item.id !== targetId));

// Updating one object field
setUser(prev => ({ ...prev, name: 'New Name' }));
```

3. State with Arrays — The Patterns

Array State Patterns

```
const [items, setItems] = useState([]);

// ADD an item
setItems(prev => [...prev, newItem]);

// REMOVE an item
setItems(prev => prev.filter(i => i.id !== id));

// UPDATE one item
setItems(prev => prev.map(i =>
  i.id === id ? { ...i, done: true } : i
));

// SORT items
setItems(prev => [...prev].sort((a,b) => a.name.localeCompare(b.name)));
```

4. Lazy Initialization

If computing the initial state is expensive (reading from `localStorage`, filtering a big array), use a function so it only runs once:

Lazy Initialization

```
// ■ Runs readFromStorage on EVERY render
const [data, setData] = useState(readFromStorage());

// ■ Runs readFromStorage only ONCE
const [data, setData] = useState(() => readFromStorage());
```

5. Lifting State Up

When two sibling components need the same data, lift the state to their nearest common parent:

Lifting State Up

```
// Sibling A and B both need 'theme'
// Lift it to the parent:

function App() {
  const [theme, setTheme] = useState('light');

  return (
    <>
      <ThemeToggle theme={theme} setTheme={setTheme} />
      <MainContent theme={theme} />
    </>
  );
}
```

■ TIP

Ask: 'Which component owns this data?'

If TWO components need it → lift to their common parent.

This is one of the most important React patterns!

6. ■■ Mini Project — Expense Calculator

■■ MINI PROJECT

Build an expense tracker:

- Add expenses with name and amount
- Remove individual expenses
- Show total, average, highest expense
- Filter by category

Time: 30–45 minutes. Practice ALL useState array patterns.

ExpenseTracker.jsx

```
function ExpenseTracker() {
  const [expenses, setExpenses] = useState([]);
  const [name, setName] = useState('');
  const [amount, setAmount] = useState('');

  function addExpense() {
    if (!name || !amount) return;
    setExpenses(prev => [
      ...prev,
      { id: Date.now(), name, amount: parseFloat(amount) }
    ]);
    setName('');
    setAmount('');
  }

  function removeExpense(id) {
    setExpenses(prev => prev.filter(e => e.id !== id));
  }

  const total = expenses.reduce((sum, e) => sum + e.amount, 0);

  return (
    <div>
      <input value={name} onChange={e=>setName(e.target.value)} />
      <input value={amount} onChange={e=>setAmount(e.target.value)}
        type='number' />
      <button onClick={addExpense}>Add</button>
      <p>Total: ${total.toFixed(2)}</p>
      {expenses.map(e => (
        <div key={e.id}>
          {e.name}: ${e.amount}
          <button onClick={()=>removeExpense(e.id)}>X</button>
        </div>
      ))}
    </div>
  );
}
```

7. Exercises

[Easy] Build a stack (push/pop): add words to a list, remove the most recent one with 'Undo'.

[Easy] Create a 'favourites' button on 5 items. Each can be toggled. Show total favourites count.

[Easy] Build a quantity selector: + and - buttons, minimum 0, maximum 10. Show price = quantity × \$9.99.

[Medium] Build a shopping list: add items, check them off (strikethrough), remove checked items all at once.

[Medium] Create a score leaderboard: add player names & scores, sort by score, highlight #1 player in gold.

[Challenge] Build a Kanban board with 3 columns (To Do, In Progress, Done). Cards can move between columns.

8. Interview Questions

Q: Why should you use the functional form of `setState (prev => ...)`?

A: When new state depends on old state, the functional form guarantees you're working with the most recent value. This prevents bugs in async scenarios where state updates might be batched.

Q: What does 'lifting state up' mean?

A: Moving state to the nearest common ancestor of the components that need it. This allows sibling components to share and sync the same data by receiving it as props from the parent.

Q: How do you add an item to an array in state?

A: Use the spread operator to create a new array: `setItems(prev => [...prev, newItem])`. Never push directly to the existing array — React needs a new reference to detect the change.

9. Lesson Summary

■ KEY TAKEAWAY

Use functional updates (`prev => ...`) when new state depends on old state.

■ KEY TAKEAWAY

Array patterns: spread to add, filter to remove, map to update.

■ KEY TAKEAWAY

Lazy init: pass a function to `useState` for expensive initial values.

■ KEY TAKEAWAY

Lifting state up = moving state to a common parent so siblings can share it.

■ KEY TAKEAWAY

NEVER mutate state directly — always create a new array/object.

LESSON 08

useEffect — Side Effects & Lifecycle

1. Why useEffect Exists

React components are supposed to be 'pure' — given the same inputs, they return the same output, with no surprises. But real apps need to do 'impure' things: fetch data, set timers, update the document title, listen to events.

■ ANALOGY

useEffect is like the 'after class' time.

During class (rendering), the teacher just teaches.

After class (useEffect), students can do homework, call friends, and go to after-school clubs.

useEffect runs AFTER React has updated the screen.

2. The Three Forms of useEffect

The Three useEffect Forms

```
// FORM 1: No dependency array – runs after EVERY render
useEffect(() => {
  console.log('I run after every render');
});
```

```
// FORM 2: Empty [] – runs ONCE after first render
useEffect(() => {
  console.log('I run only once, like componentDidMount');
  fetchData(); // Good place for initial data fetching!
}, []);
```

```
// FORM 3: [dependency] – runs when dependency changes
useEffect(() => {
  console.log('userId changed, fetch new profile');
  fetchProfile(userId);
}, [userId]); // runs when userId changes
```

■ TIP

The dependency array controls WHEN useEffect runs.

No array → runs every render (usually wrong)

Empty [] → runs once on mount

[value] → runs when value changes

3. Cleanup Functions

When `useEffect` sets up something ongoing (a timer, event listener, subscription), you must clean it up when the component unmounts or the effect re-runs:

Cleanup Functions

```
useEffect(() => {
  // SET UP: start a timer
  const timer = setInterval(() => {
    setTime(t => t + 1);
  }, 1000);

  // CLEAN UP: runs before next effect OR on unmount
  return () => clearInterval(timer);
}, []);

// Same pattern for event listeners:
useEffect(() => {
  window.addEventListener('resize', handleResize);
  return () => window.removeEventListener('resize', handleResize);
}, []);
```

4. Common `useEffect` Patterns

Common `useEffect` Patterns

```
// Pattern 1: Fetch data on mount
useEffect(() => {
  fetch('/api/users')
    .then(r => r.json())
    .then(data => setUsers(data));
}, []);

// Pattern 2: React to a prop/state change
useEffect(() => {
  document.title = `${count} notifications`;
}, [count]);

// Pattern 3: Save to localStorage automatically
useEffect(() => {
  localStorage.setItem('theme', theme);
}, [theme]);
```

5. Common Beginner Mistakes

■ COMMON MISTAKE

Infinite loop: putting a value in deps that the effect itself updates — the effect fires, updates state, which fires the effect again forever.

■ COMMON MISTAKE

Missing dependencies: if you use a variable inside `useEffect` but don't put it in the `deps` array, you get stale (outdated) values.

■ COMMON MISTAKE

Not cleaning up: forgetting `return () => cleanup()` causes memory leaks and weird bugs when navigating between pages.

■ COMMON MISTAKE

Putting `async` directly in `useEffect`: `useEffect(async () => {})` causes a warning. Define `async` function inside and call it.

Async in `useEffect` – Correct Pattern

```
// ■ Correct async pattern in useEffect
useEffect(() => {
  async function loadData() {
    const res = await fetch('/api/data');
    const data = await res.json();
    setData(data);
  }
  loadData();
}, []);
```

6. ■■ Mini Project — Auto Theme Saver

■■ MINI PROJECT

Build a theme switcher that remembers your choice:

- Toggle between light/dark mode
- `useEffect` saves theme to `localStorage` on every change
- On load (`useEffect` with `[]`), read saved theme from `localStorage`
- Update `document.body.className` to apply the theme

Time: 20–30 minutes.

ThemeSwitcher.jsx

```
function ThemeSwitcher() {
  const [theme, setTheme] = useState(() => {
    return localStorage.getItem('theme') || 'light';
  });

  useEffect(() => {
    document.body.className = theme;
    localStorage.setItem('theme', theme);
  }, [theme]);

  return (
    <button onClick={() => setTheme(t => t==='light'? 'dark': 'light')}>
      Switch to {theme === 'light' ? 'Dark' : 'Light'} Mode
    </button>
  );
}
```

7. Exercises

[Easy] Change the document title to show a count that updates every second using `setInterval` in `useEffect`.

[Easy] Fetch a random joke from https://official-joke-api.appspot.com/random_joke and display it on mount.

[Easy] Build a 'time alive' counter: shows how many seconds since the component mounted.

[Medium] Build a search box that fetches results from an API 500ms after the user stops typing (debounce with `setTimeout/clearTimeout`).

[Medium] Create a window size display that updates in real-time using a `resize` event listener in `useEffect`.

[Challenge] Build a Pokémon viewer: input for Pokémon name, `useEffect` fetches from the PokéAPI when name changes, shows image, stats, and type.

8. Interview Questions

Q: What is `useEffect` used for?

A: `useEffect` runs code after rendering — for side effects like API calls, subscriptions, timers, and syncing with external systems. It's React's way of handling things that happen 'outside' of rendering.

Q: What does the dependency array in `useEffect` do?

A: It controls when the effect runs. Empty `[]` = once on mount. `[value]` = every time value changes. No array = every render.

Q: What is a cleanup function in `useEffect` and when do you need it?

A: The function returned from `useEffect`. It runs before the next effect execution or when the component unmounts. You need it to clear timers, remove event listeners, and cancel subscriptions to prevent memory leaks.

9. Lesson Summary

■ KEY TAKEAWAY

useEffect runs after render for side effects (API, timers, events).

■ KEY TAKEAWAY

No deps = every render. [] = once. [x] = when x changes.

■ KEY TAKEAWAY

Always return a cleanup function when setting up timers or listeners.

■ KEY TAKEAWAY

Use an inner async function for async code inside useEffect.

■ KEY TAKEAWAY

Wrong dependencies = stale values or infinite loops. Read errors carefully.

LESSON 09

React Router — Multi-Page Apps

1. Why React Router Exists

React by default is a Single Page Application (SPA). There's only ONE HTML file. But users expect different URLs for different pages — /home, /about, /products/42.

■ ANALOGY

React Router is like a hotel receptionist.

A guest arrives and says 'I want room /about'.

The receptionist (Router) reads the URL and shows the right room (component).

The building never changes — just what you see inside changes.

2. Installation & Setup

Setup React Router

```
# Install React Router
npm install react-router-dom

# In your index.jsx or main.jsx:
import { BrowserRouter } from 'react-router-dom';

ReactDOM.createRoot(document.getElementById('root')).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

3. Core Concepts

Basic Router Setup

```
import { Routes, Route, Link, useNavigate, useParams }
  from 'react-router-dom';

// App.jsx – define your routes
function App() {
  return (
    <div>
      {/* Navigation */}
      <nav>
        <Link to='/'>Home</Link>
        <Link to='/about'>About</Link>
        <Link to='/contact'>Contact</Link>
      </nav>

      {/* Route definitions */}
      <Routes>
        <Route path='/'      element={<HomePage />} />
        <Route path='/about' element={<AboutPage />} />
        <Route path='/contact' element={<ContactPage />} />
        <Route path='*'      element={<NotFoundPage />} />
      </Routes>
    </div>
  );
}
```

4. URL Parameters — Dynamic Routes

URL Parameters with useParams

```
// Route with a parameter
<Route path='/product/:id' element={<ProductPage />} />

// ProductPage.jsx – reading the parameter
import { useParams } from 'react-router-dom';

function ProductPage() {
  const { id } = useParams();

  return <h1>Showing product #{id}</h1>;
  // Visit /product/42 → Shows: Showing product #42
}
```

5. Programmatic Navigation

Programmatic Navigation

```
import { useNavigate } from 'react-router-dom';

function LoginPage() {
  const navigate = useNavigate();

  function handleLogin() {
    // after successful login:
    navigate('/dashboard');

    // Go back:
    navigate(-1);

    // Replace history (no back button):
    navigate('/home', { replace: true });
  }

  return <button onClick={handleLogin}>Login</button>;
}
```

6. Nested Routes

Nested Routes with Outlet

```
// Dashboard with sub-pages
<Route path='/dashboard' element={<Dashboard />}>
  <Route index element={<Overview />} />
  <Route path='profile' element={<Profile />} />
  <Route path='settings' element={<Settings />} />
</Route>

// Dashboard.jsx – renders child routes here
import { Outlet } from 'react-router-dom';

function Dashboard() {
  return (
    <div>
      <Sidebar />
      <Outlet /> { /* child route renders here */ }
    </div>
  );
}
```

7. Common Beginner Mistakes

■ COMMON MISTAKE

Using `<a href='...'>` instead of `<Link to='...'>`. The anchor tag refreshes the whole page!

■ COMMON MISTAKE

Forgetting to wrap the app in `<BrowserRouter>` — nothing will work without it.

■ COMMON MISTAKE

Using `<Route>` outside `<Routes>` — always wrap routes in `<Routes>`.

■ COMMON MISTAKE

Expecting `useParams` to work on a component not inside a `<Route>`.

8. ■■ Mini Project — Multi-Page Portfolio

■■ MINI PROJECT

Build a 5-page portfolio site with React Router:

`/` → Home page with intro

`/about` → Your story and skills

`/projects` → Project grid (with `<ProjectCard />` components)

`/projects/:id` → Individual project detail page

`/contact` → Contact form

Add a sticky Navbar with `<Link>` to each page.

9. Exercises

[Easy] Create a 3-page app: Home, About, Contact. Add a nav bar with Links to each.

[Easy] Add a 404 page that shows when users visit an unknown URL (use `path=*`).

[Easy] Use `useNavigate` to redirect to `/home` after clicking a 'Get Started' button.

[Medium] Build a blog with a `/blog` route listing 5 posts, and `/blog/:id` showing each post's full content.

[Medium] Create a dashboard with nested routes: `/dash/overview`, `/dash/settings`, `/dash/profile` sharing a sidebar.

[Challenge] Build a protected route: `/admin` only shows if a user is 'logged in' (stored in state). Otherwise redirect to `/login`.

10. Interview Questions

Q: What is React Router and why do we need it?

A: React Router is a library that enables navigation in single-page applications. Without it, React apps have only one URL. Router maps URLs to components so each page has its own address.

Q: What is the difference between Link and a regular anchor tag?

A: Link from react-router-dom navigates without a full page reload, preserving React state. A regular anchor tag causes a full browser page refresh, resetting everything.

Q: What is `useParams` used for?

A: `useParams` is a hook that lets you access dynamic URL parameters. If a route is `/product/:id` and you visit `/product/42`, `useParams()` returns `{ id: '42' }`.

11. Lesson Summary

■ KEY TAKEAWAY

Wrap your app in `<BrowserRouter>` to enable routing.

■ KEY TAKEAWAY

Use `<Routes>` + `<Route>` to define URL-to-component mappings.

■ KEY TAKEAWAY

Use `<Link>` for navigation — NEVER `<a href>` in React.

■ KEY TAKEAWAY

`useParams` reads dynamic segments from the URL (`:id`, `:slug`).

■ KEY TAKEAWAY

`useNavigate` lets you redirect programmatically (after login, submit, etc.).

LESSON 10

API Fetching — Real Data, Real Apps

1. Why API Fetching Matters

So far your apps have used hardcoded data. Real apps connect to servers to get live data — a social media feed, product prices, weather updates, user profiles.

■ ANALOGY

Your component is a restaurant customer.

The API is the kitchen. Your `fetch()` call is ordering food.

While the kitchen prepares (loading state)...

The food arrives (data state) or something goes wrong (error state).

2. The Standard Fetch Pattern

The Standard Fetch Pattern (Loading/Error/Data)

```
import { useState, useEffect } from 'react';

function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function fetchUsers() {
      try {
        const res = await fetch('https://jsonplaceholder.typicode.com/users');
        if (!res.ok) throw new Error('Failed to fetch');
        const data = await res.json();
        setUsers(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }
    fetchUsers();
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <ul>
      {users.map(u => <li key={u.id}>{u.name}</li>)}
    </ul>
  );
}
```

3. The Three States of Fetching

■ MENTAL MODEL

Every API call has 3 possible states:

1. LOADING — waiting for data (show spinner or skeleton)
2. ERROR — something went wrong (show error message)
3. SUCCESS — data arrived (show the data)

Always handle all three! Users need feedback.

4. POST Requests — Sending Data

POST Requests

```
async function createPost(title, body) {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title, body, userId: 1 }),
  });

  const data = await res.json();
  console.log('Created:', data);
}

// Call it from an event handler:
function Form() {
  const [title, setTitle] = useState('');

  function handleSubmit(e) {
    e.preventDefault();
    createPost(title, 'My body text');
  }

  return <form onSubmit={handleSubmit}>...</form>;
}
```

5. Refetching on Parameter Change

Refetch on Input Change

```
function PokemonViewer() {
  const [name, setName] = useState('pikachu');
  const [pokemon, setPokemon] = useState(null);
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    setLoading(true);
    async function fetchPokemon() {
      const res = await fetch(`https://pokeapi.co/api/v2/pokemon/${name}`);
      const data = await res.json();
      setPokemon(data);
      setLoading(false);
    }
    fetchPokemon();
  }, [name]); // re-runs whenever 'name' changes

  return (
    <div>
      <input value={name} onChange={e=>setName(e.target.value)} />
      {loading && <p>Fetching...</p>}
      {pokemon && (
        <div>
          <img src={pokemon.sprites.front_default} alt={name} />
          <p>#{pokemon.id} — {pokemon.name}</p>
        </div>
      )}
    </div>
  );
}
```

6. Common Beginner Mistakes

■ COMMON MISTAKE

Not handling errors — if the API is down, your app shows a blank screen with no explanation.

■ COMMON MISTAKE

Forgetting loading state — users see a flash of empty content before data loads.

■ COMMON MISTAKE

Fetching without `useEffect` — calling `fetch` directly in the component body causes infinite loops.

■ COMMON MISTAKE

Not checking `res.ok` — `fetch()` doesn't throw on 404 or 500 errors, only on network failures.

7. Free APIs to Practice With

API Name	URL	What it provides
JSONPlaceholder	https://jsonplaceholder.typicode.com	Fake posts, users, todos
PokéAPI	https://pokeapi.co	All Pokémon data
Open-Meteo	https://open-meteo.com	Weather forecast
REST Countries	https://restcountries.com	Country info & flags
The Dog API	https://thedogapi.com	Dog images & breeds

8. ■■ Mini Project — Pokémon Viewer

■■ MINI PROJECT

Build a Pokémon browser using the PokéAPI:

- Search by name with a text input
- Show image, name, ID, height, weight
- Display type badges (fire, water, etc.) with colours
- Loading spinner while fetching
- Error message for invalid Pokémon names
- Prev/Next navigation to browse by ID

Time: 30–45 minutes. This is a real portfolio piece!

9. Exercises

[Easy] Fetch a list of users from JSONPlaceholder and display their names and emails in a table.

[Easy] Fetch a random dog image on button click from <https://dog.ceo/api/breeds/image/random>

[Easy] Show a loading spinner (or text) while any fetch is in progress.

[Medium] Build a country search: type a country name, show its flag, capital, and population using REST Countries API.

[Medium] Build an autocomplete search that fetches suggestions from an API as the user types (with debouncing).

[Challenge] Build a GitHub user profile viewer: enter a username, show their avatar, bio, repos count, and list of their latest 5 repositories.

10. Interview Questions

Q: How do you fetch data in React?

A: Use the `fetch()` API (or `axios`) inside a `useEffect` hook with an empty dependency array `[]` so it runs once on mount. Store the result in state. Always handle loading and error states.

Q: Why can't you make `useEffect` async directly?

A: `useEffect` expects a cleanup function or nothing to be returned. Async functions return Promises, which React doesn't know how to use as cleanup. Instead, define an async function inside and call it.

Q: What is the difference between a 404 error and a network error in `fetch`?

A: `fetch()` only throws (rejects) on network failures (no internet, DNS error). A 404 or 500 response still resolves successfully — you must check `response.ok` or `response.status` to detect HTTP errors.

11. Lesson Summary

■ **KEY TAKEAWAY**

Fetch data inside `useEffect` with an empty `[]` dependency array.

■ **KEY TAKEAWAY**

Always manage three states: loading, error, and data.

■ **KEY TAKEAWAY**

Check `res.ok` — `fetch` doesn't throw on 404/500 errors.

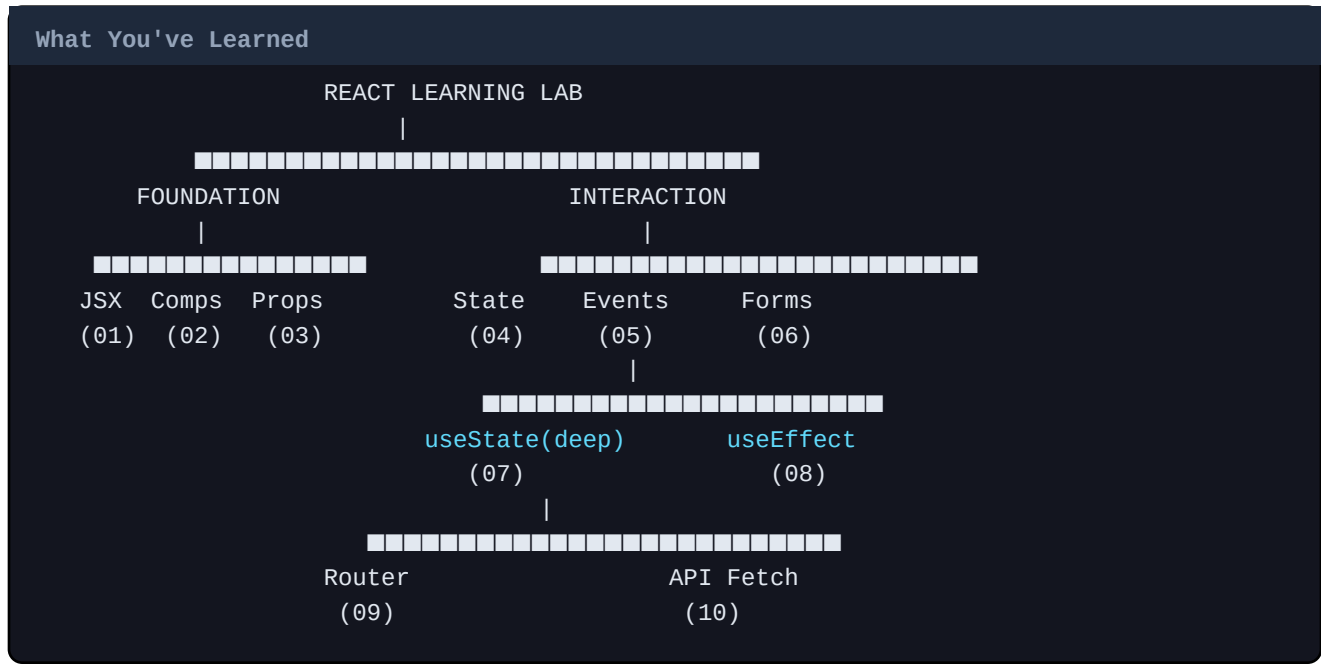
■ **KEY TAKEAWAY**

POST requests need `method`, `headers`, and `JSON.stringify(body)`.

■ **KEY TAKEAWAY**

Refetch by including the search param in the `deps` array.

Here's everything you've learned and where it fits in the big picture:



What You've Mastered

Concept	What you can do now
JSX	Writing UI in JavaScript — the language of React.
Components	Building reusable, independent pieces of UI.
Props	Passing data from parent to child components.
State	Giving components memory that triggers re-renders.
Events	Responding to user clicks, typing, and more.
Forms	Collecting and validating user input.
useState	All the array/object patterns for real data.
useEffect	Side effects: APIs, timers, localStorage, events.
Router	Multi-page navigation without page refreshes.
API Fetch	Loading real data from backend servers and APIs.

What's Next for Internships

Topic	Difficulty	Why It Matters
useContext	Medium	Share state across many components without prop-drilling.
useRef	Medium	Access DOM elements directly and persist values without re-renders.
Custom Hooks	Medium	Extract reusable logic into your own hooks.
Tailwind CSS	Easy	Utility-first CSS framework used in most modern projects.
TypeScript	Hard	Typed JavaScript — most companies require this.
React Query / SWR	Medium	Better data fetching with caching and auto-refresh.
Zustand / Redux	Hard	Global state management for large applications.
Next.js	Medium	React framework with routing, SSR, and deployment built in.
Vite	Easy	Fast build tool — already using it if you used Vite setup.
Testing (Vitest)	Hard	Writing tests for components — highly valued by employers.

■ TIP

You now have 80% of what you need for a React internship.
The remaining 20% (TypeScript, testing, Next.js) takes weeks not days.
Build projects FIRST. Learn the rest as you need it.

Movie Night Planner — A Social Watchlist App

■ ■ MINI PROJECT

Not a To-Do app. Not a weather app. Not a calculator.
Movie Night Planner: plan, rate, and share your movie watchlist
with everything you've learned packed into one real-world project.
A recruiter WILL explore this — make it shine.

What You're Building

A multi-page React app where users can browse movies from the TMDB API, build a personal watchlist, rate movies they've seen, and plan a movie night with a friend.

Pages & Features

/

Home — Hero + trending movies fetched from TMDB

■ TIP

Concepts practiced: API Fetching, useEffect, Components, Props

/browse

Browse — Search movies by name, filter by genre

■ TIP

Concepts practiced: useState, useEffect, Forms, Events

/watchlist

Watchlist — Your saved movies, mark as Watched

■ TIP

Concepts practiced: useState (arrays), localStorage via useEffect

/movie/:id

Movie Detail — Full info, trailer, cast, 'Add to Watchlist'

■ TIP

Concepts practiced: React Router (useParams), API Fetching

/planner

Movie Night Planner — Pick date, pick movie, invite friend name

■ TIP

Concepts practiced: Forms, useState, Events, JSX

/ratings

My Ratings — Rate watched movies 1–5 stars, add notes

■ TIP

Concepts practiced: useState (objects), Forms, Components

Component Architecture

Project Structure

```
src/
├── components/
│   ├── Navbar.jsx           # Links to all pages
│   ├── MovieCard.jsx        # Reusable card (props: title, poster, rating)
│   ├── MovieGrid.jsx        # Maps over movies array → MovieCards
│   ├── SearchBar.jsx        # Controlled input + onChange
│   ├── StarRating.jsx       # 1-5 star interactive rater
│   ├── WatchlistButton.jsx   # Add/remove with visual feedback
│   └── LoadingSpinner.jsx    # Shows while fetching
├── pages/
│   ├── HomePage.jsx
│   ├── BrowsePage.jsx
│   ├── WatchlistPage.jsx
│   ├── MovieDetailPage.jsx
│   ├── PlannerPage.jsx
│   └── RatingsPage.jsx
├── hooks/
│   └── useMovies.js          # Custom hook for fetching movies
└── App.jsx                  # Router setup
```

Concept Checklist

Concept	Where it appears in Movie Night Planner
JSX	Every component file — writing UI declaratively
Components	MovieCard, Navbar, StarRating, LoadingSpinner...
Props	MovieCard receives title, poster, rating, onAdd as props
State	watchlist[], ratings{}, searchQuery, selectedGenre
Events	onClick (add/remove), onChange (search), onSubmit (planner)
Forms	Movie Night Planner form with date, movie, friend name
useState	Watchlist array with add/remove/update patterns
useEffect	Fetch trending on mount, refetch when search changes
Router	6 pages with <Link>, useParams on /movie/:id
API Fetch	TMDB API — search, movie details, trending

Starter Code — App.jsx

App.jsx – Full Starter

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import { useState } from 'react';
import Navbar      from './components/Navbar';
import HomePage    from './pages/HomePage';
import BrowsePage  from './pages/BrowsePage';
import WatchlistPage from './pages/WatchlistPage';
import MovieDetailPage from './pages/MovieDetailPage';
import PlannerPage from './pages/PlannerPage';
import RatingsPage from './pages/RatingsPage';

function App() {
  const [watchlist, setWatchlist] = useState([]);
  const [ratings,   setRatings]   = useState({});

  function addToWatchlist(movie) {
    setWatchlist(prev => {
      if (prev.find(m => m.id === movie.id)) return prev;
      return [...prev, movie];
    });
  }

  function rateMovie(id, score) {
    setRatings(prev => ({ ...prev, [id]: score }));
  }

  return (
    <BrowserRouter>
      <Navbar watchlistCount={watchlist.length} />
      <Routes>
        <Route path="/"      element={<HomePage add={addToWatchlist} />} />
        <Route path="/browse" element={<BrowsePage add={addToWatchlist} />} />
        <Route path="/watchlist" element={<WatchlistPage list={watchlist}
                                                    setList={setWatchlist}
                                                    ratings={ratings} />} />
        <Route path="/movie/:id" element={<MovieDetailPage add={addToWatchlist} />} />
        <Route path="/planner"  element={<PlannerPage list={watchlist} />} />
        <Route path="/ratings"  element={<RatingsPage list={watchlist}
                                                    ratings={ratings}
                                                    onRate={rateMovie} />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Getting a TMDB API Key

1. Go to <https://www.themoviedb.org> and create a free account.
2. Go to Settings → API → Request an API key.
3. Choose 'Developer' — it's free and instant.
4. Store it as an environment variable:

TMDB API Key Setup

```
# .env file (in project root)
VITE_TMDB_KEY=your_api_key_here

# In your component:
const API_KEY = import.meta.env.VITE_TMDB_KEY;
const URL = `https://api.themoviedb.org/3/trending/movie/week?api_key=${API_KEY}`;
```

Bonus Stretch Goals

- Dark/Light mode toggle with useEffect + localStorage
- Movie night calendar: mark dates with planned movies
- Share watchlist via URL (encode to query string)
- Add animations with CSS transitions on card hover
- Filter watchlist by genre, year, or rating
- Random movie picker button (pick from watchlist randomly)

■ KEY TAKEAWAY

Congratulations! You've completed the React 80/20 Course.
You can now build REAL React applications that companies actually ship.
Build the Movie Night Planner. Put it on GitHub. Add it to your resume.
You're internship-ready. Go get that job. ■

Made with ❤️■ for curious builders. The best way to learn is to BUILD.